In [286…

```
import sys
import os
import importlib

#-------------------------------------------------------------
# clearing cache so that the core.py is called from the right place, had
modules_to_clear = [
    m for m in sys.modules.keys()
    if any(p in m for p in ["models", "src", "data", "metrics"])
]
for module in modules_to_clear:
    del sys.modules[module]
PROJECT_ROOT = "/Users/darya/Desktop/Diploma/COMPAS - criminals/whole_pro
sys.path = [p for p in sys.path if "COMPAS - criminals" not in p]
sys.path.insert(0, PROJECT_ROOT)
cox_upd_path = os.path.join(PROJECT_ROOT, "models", "cox_model_upd.py")
spec = importlib.util.spec_from_file_location("cox_model_upd", cox_upd_pa
cox_model_upd = importlib.util.module_from_spec(spec)
spec.loader.exec_module(cox_model_upd)
CoxModelUpd = cox_model_upd.CoxModelUpd
#issue with the ibs_metric part because of numba, i beleive this helped f
os.environ["NUMBA_DISABLE_JIT"] = "1"
#-------------------------------------------------------------

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

from data.data_processor import DataProcessor
from data.splitter import SurvivalSplitter
from models.cox_model import CoxModel
from models.cox_model_upd import CoxModelUpd
from metrics.ibs import IBSMetric
from metrics.ibs_remain import IBSRemainMetric
from metrics.auprc import AUPRCMetric
from metrics.recurrent_count_error import RecurrentCountError
from pipeline.evaluator import SurvivalEvaluator
from survivors.constants import get_y
```

In [287…

```
df = pd.read_csv('data/individual_custody_timeline_rfm.csv')
print(df.columns)
```

```
Index(['name', 'age', 'sex', 'race', 'c_charge_degree', 'c_charge_desc',
       'in_custody', 'out_custody', 'custody_num', 'time_to_last_custody',
       'time_since_last_arrest', 'time_to_future_arrest', 'event',
       'event_of_future_arrest', 'curr_dur_custody', 'sum_dur_custody',
       'average_dur_custody', 'all_future_arrest_times', 'all_future_event
s'],
      dtype='object')
```

In [288…

```
df
```

Out[288…

	name	age	sex	race	c_charge_degree	c_charge_desc	in_cu
0	aajah herrington	23	Female	African-American	(F3)	Driving While License Revoked	201
1	aajah herrington	23	Female	African-American	(F3)	Driving While License Revoked	201
2	aaliyah lovo	21	Female	Caucasian	(F2)	Burglary Unoccupied Dwelling	2013-
3	aaron aprile	27	Male	Hispanic	(F3)	Grand Theft in the 3rd Degree	201
4	aaron aprile	27	Male	Hispanic	(F3)	Grand Theft in the 3rd Degree	2014-
...	...	...	...	...	...	...	
18663	zolya isale	21	Female	African-American	NaN	NaN	20
18664	zuberi floyd	22	Male	African-American	(F3)	Uttering a Forged Instrument	2013
18665	zuberi floyd	22	Male	African-American	(F3)	Uttering a Forged Instrument	20
18666	zulimay lizcano	44	Female	Caucasian	(F3)	Grand Theft in the 3rd Degree	20
18667	zulma ramos	40	Female	Hispanic	(M2)	arrest case no charge	201

18668 rows × 19 columns

In [289…

```
# importlib.reload(data.data_processor)

processor = DataProcessor("data/individual_custody_timeline_rfm.csv")
cox_df = processor.load_and_prepare()
'''cox_df = pd.get_dummies(
    cox_df,
    columns=["sex", "race"],
    drop_first=True
)'''
cox_df.head()
```

Out[289…

	name	episode_col	entry	dur	event	age	time_since_last_arrest	averag
0	aajah herrington	1	0.0	385.0	0	23	384.0	
1	aaron aprile	1	0.0	55.0	0	27	0.0	
2	aaron davis	1	0.0	432.0	0	41	314.0	
3	aaron eddins	1	0.0	104.0	0	37	104.0	
4	aaron epstein	1	0.0	297.0	1	22	296.0	

Data split into train and test data

In [290…

```
splitter = SurvivalSplitter(test_size=0.2)
train_df, test_df = splitter.split_by_individual(cox_df)
train_df = train_df.drop(columns=["curr_dur_custody"])
test_df = test_df.drop(columns=["curr_dur_custody"])

print("Train individuals:", train_df["name"].nunique())
print("Test individuals:", test_df["name"].nunique())
```

```
Train individuals: 3382
Test individuals: 845
```

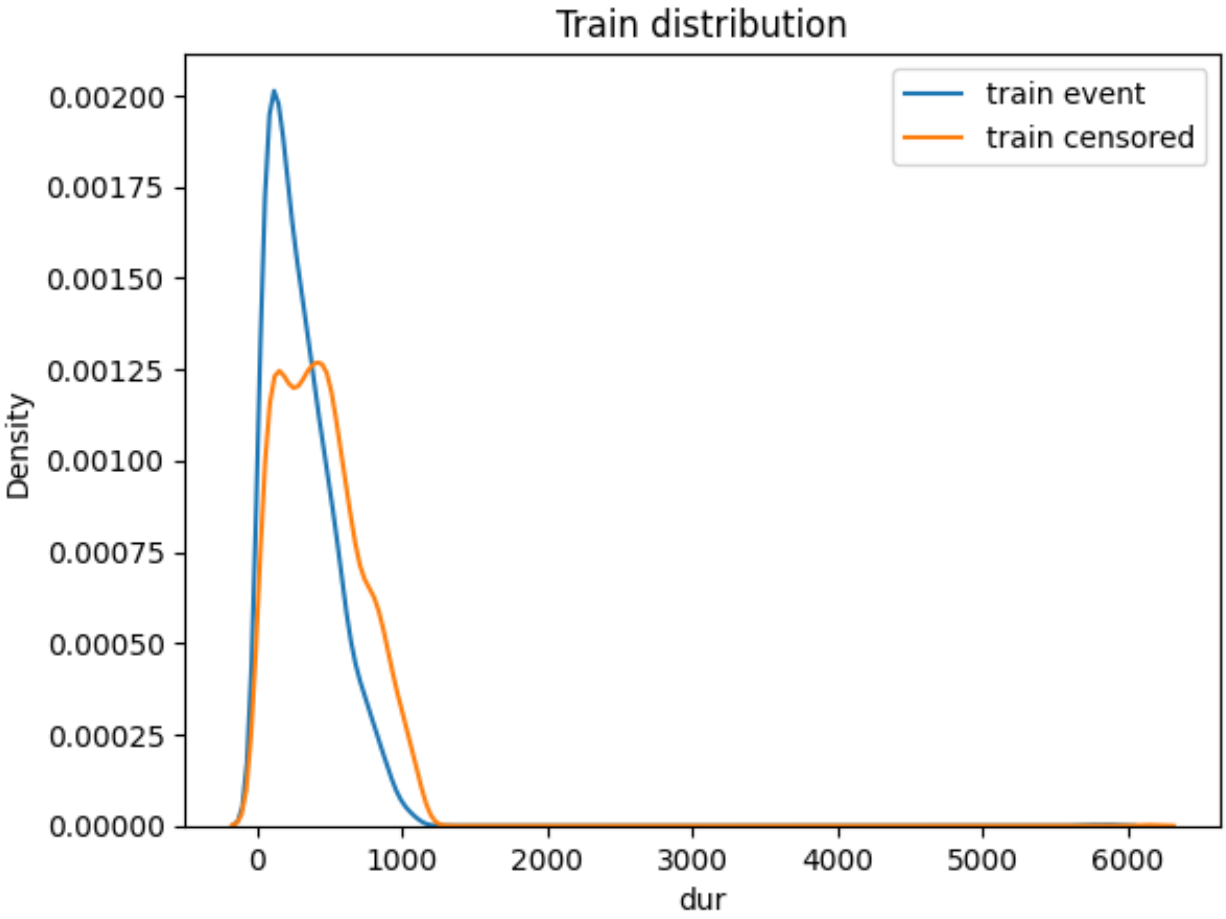
In [291…

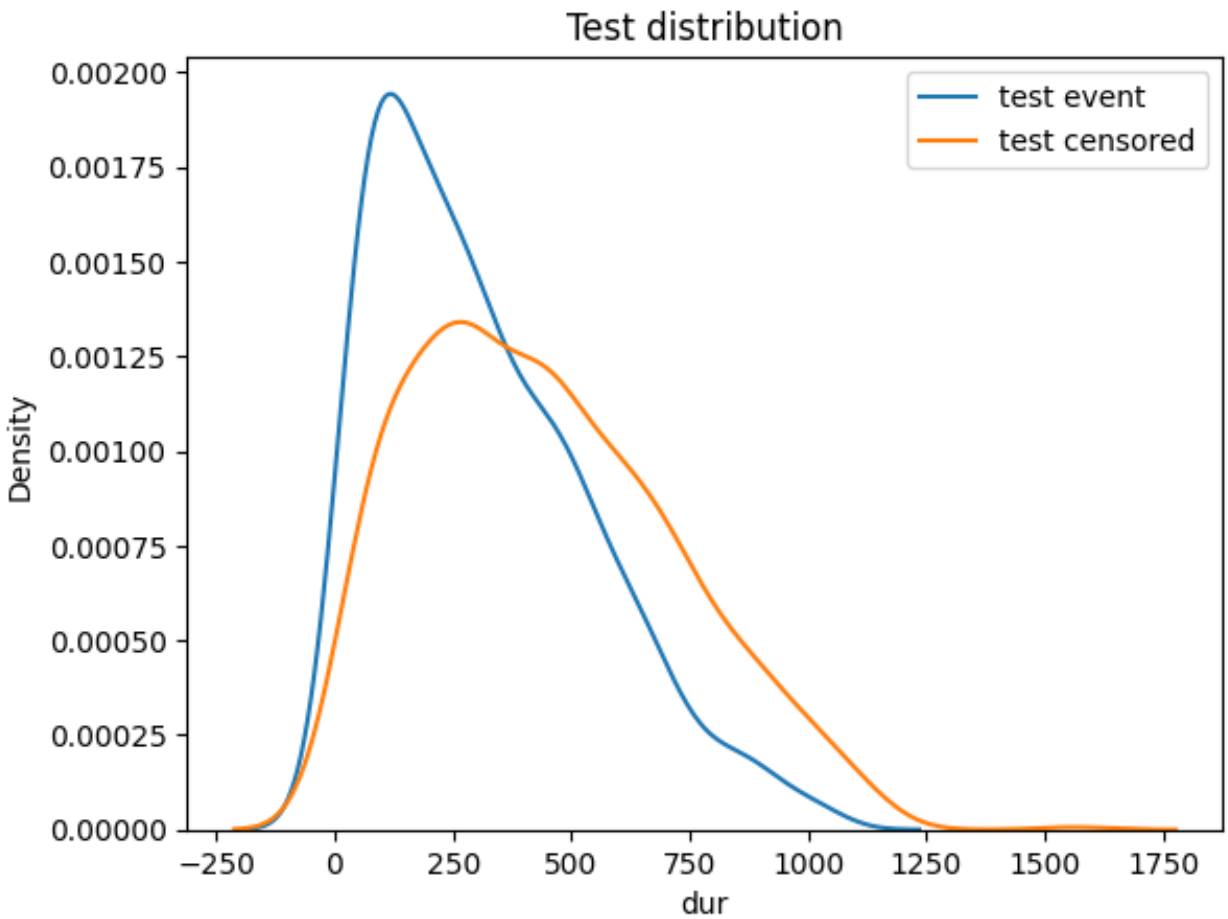
```
train_df["time"] = train_df["entry"] + train_df["dur"]
test_df["time"] = test_df["entry"] + test_df["dur"]

train_cens = train_df["event"].astype(bool)
test_cens = test_df["event"].astype(bool)

survival_train = get_y(
    cens=train_cens,
    time=train_df["time"],
    competing=False
)

survival_test = get_y(
    cens=test_cens,
    time=test_df["time"],
    competing=False
)
```

In [292…

```
plt.figure()
sns.kdeplot(train_df[train_df["event"]==1]["dur"], label="train event")
sns.kdeplot(train_df[train_df["event"]==0]["dur"], label="train censored"
plt.legend()
plt.title("Train distribution")
plt.show()

plt.figure()
sns.kdeplot(test_df[test_df["event"]==1]["dur"], label="test event")
```

```
sns.kdeplot(test_df[test_df["event"]==0]["dur"], label="test censored")
plt.legend()
plt.title("Test distribution")
plt.show()

print("Train events:", train_df["event"].value_counts())
print("Test events:", test_df["event"].value_counts())
```






```
Train events: event
0    3382
1    2767
Name: count, dtype: int64
Test events: event
0    845
1    778
Name: count, dtype: int64
```

Using the original CoxModel file that I had (cox_model.py)

In [293…

```python
# importlib.reload(models.cox_model)

print(train_df.columns)

features = [c for c in train_df.columns
            if c not in ["name", "entry", "time", "event", "dur", "episod
print(features)

cols_for_model = ["name", "entry", "time", "event"] + features
train_model_df = train_df[cols_for_model].copy()


model = CoxModel(features=features)
model.fit(train_model_df)
```

```
Index(['name', 'episode_col', 'entry', 'dur', 'event', 'age',
       'time_since_last_arrest', 'average_dur_custody', 'time'],
      dtype='object')
['age', 'time_since_last_arrest', 'average_dur_custody']
```

model	lifelines.CoxPHFitter
duration col	'time'
event col	'event'
entry col	'entry'
cluster col	'name'
robust variance	True
baseline estimation	breslow
number of observations	6149
number of events observed	2767
partial log-likelihood	-20405.07
time fit was run	2026-03-04 12:44:28 UTC

	coef	exp(coef)	se(coef)	coef lower 95%	coef upper 95%	exp(coef) lower 95%	exp(coef upper 95%
age	-0.00	1.00	0.00	-0.01	-0.00	0.99	1.0
time_since_last_arrest	-0.00	1.00	0.00	-0.00	-0.00	1.00	1.0
average_dur_custody	-0.00	1.00	0.00	-0.00	-0.00	1.00	1.0

Concordance	0.74
Partial AIC	40816.15
log-likelihood ratio test	1166.16 on 3 df
-log2(p) of ll-ratio test	836.44

```
<class 'lifelines.fitters.coxph_fitter.CoxPHFitter'>
```

In [294…

```
train_df["dur"].describe()
```

Out[294…

```
count    6149.000000
mean      381.495853
std       289.923100
min         1.000000
25%       159.000000
50%       336.000000
75%       547.000000
max      6141.000000
Name: dur, dtype: float64
```

In [295…

```
survival_test = get_y(
    cens=test_cens,
    time=test_df["time"],
    competing=False
)
```

In [296…

```
test_df
```

Out[296…

	name	episode_col	entry	dur	event	age	time_since_last_arrest	av
0	aaron hammond	1	0.0	699.0	0	25	688.0	
1	aaron villacampa	1	0.0	107.0	0	26	106.0	
2	abraham ballestas	1	0.0	35.0	0	52	34.0	
3	acie canteen	1	0.0	215.0	0	27	214.0	
4	adam gillstrap	1	0.0	630.0	0	35	628.0	
...	...	...	...	...	...	...	...	
1618	zachary payton	1	0.0	34.0	0	28	33.0	
1619	zachary whalen	1	0.0	634.0	0	24	632.0	
1620	zaheer khan	1	0.0	184.0	0	26	183.0	
1621	zhivago walker	1	0.0	126.0	0	30	125.0	
1622	zuberi floyd	1	0.0	757.0	0	22	756.0	

1623 rows × 9 columns

In [297…

```
from lifelines.utils import concordance_index
#times = np.linspace(0, train_df["dur"].max(), 200)
horizon = np.quantile(train_model_df["time"], 0.95)
times = np.linspace(0, horizon, 200)

test_features = test_df[features].copy()
predictions = model.predict_survival(test_features, times)
#predictions = model.predict_survival(test_df, times)
estimate = predictions.values
estimate = np.asarray(estimate, dtype=np.float64)
times = np.asarray(times, dtype=np.float64)

survival_train = survival_train.astype([("event", "?"), ("time", "f8")])
survival_test = survival_test.astype([("event", "?"), ("time", "f8")])

print(survival_test.dtype.names)
ibs_metric = IBSMetric()
mean_ibs, ibs_by_time = ibs_metric.compute(
    survival_train,
    survival_test,
    estimate,
```

```
    times
)

print(mean_ibs)

survival_train_cens = survival_train.astype([("cens", "?"), ("time", "f8"
survival_test_cens = survival_test.astype([("cens", "?"), ("time", "f8")]

auprc_metric = AUPRCMetric()
auprc = auprc_metric.compute(
    survival_train_cens,
    survival_test_cens,
    estimate,
    times
)
print("AUPRC:", np.mean(auprc))

ci_metric = concordance_index(
    test_df["dur"],
    np.trapz(predictions.values, times, axis=1),
    test_cens
)
print("Concordance Index:", ci_metric)
```

```
('event', 'time')
0.2054054924976589
AUPRC: 0.5666575296287015
Concordance Index: 0.7466311236238385
```

In [298…

```
test_df
```

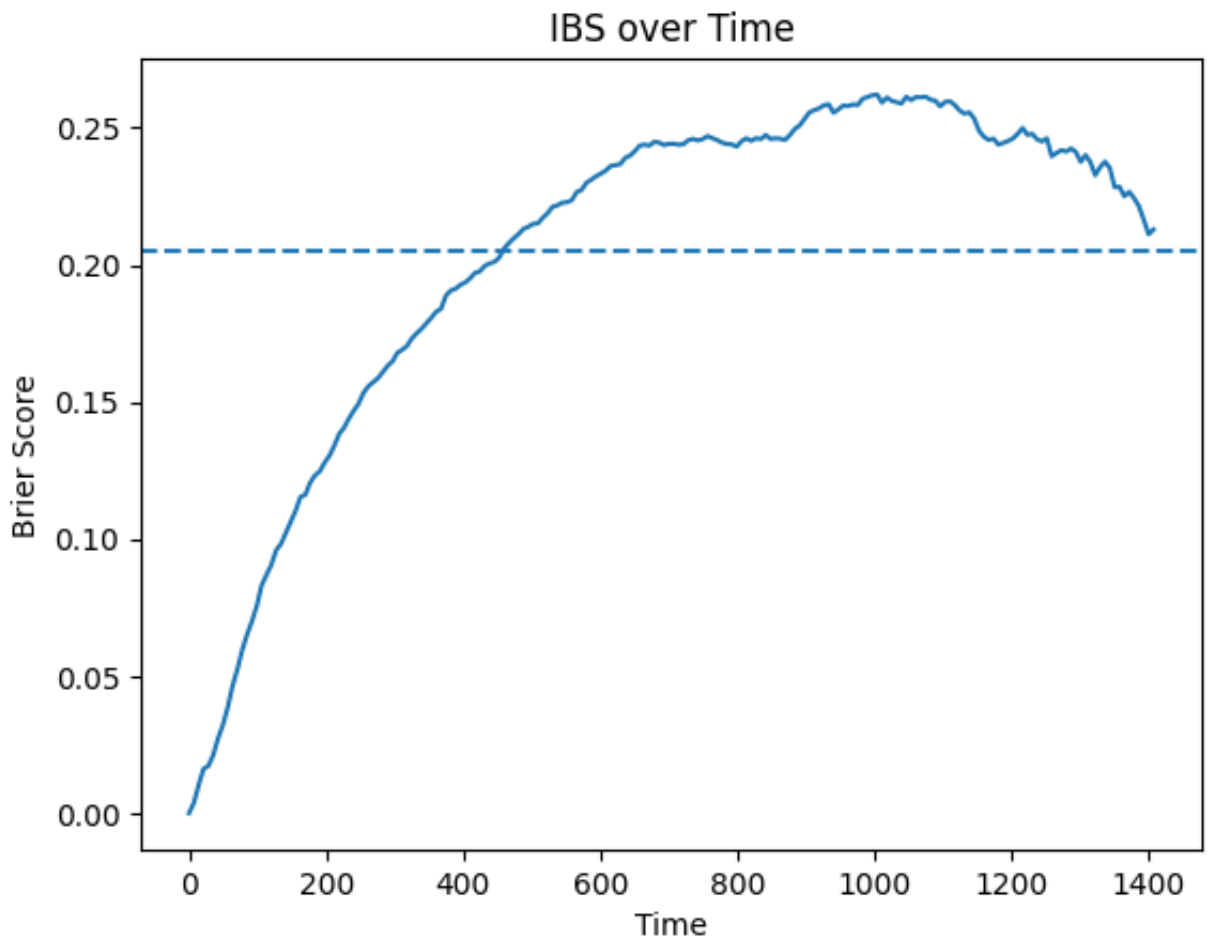
Out[298…

	name	episode_col	entry	dur	event	age	time_since_last_arrest	av
0	aaron hammond	1	0.0	699.0	0	25	688.0	
1	aaron villacampa	1	0.0	107.0	0	26	106.0	
2	abraham ballestas	1	0.0	35.0	0	52	34.0	
3	acie canteen	1	0.0	215.0	0	27	214.0	
4	adam gillstrap	1	0.0	630.0	0	35	628.0	
...	...	...	...	...	...	...	...	
1618	zachary payton	1	0.0	34.0	0	28	33.0	
1619	zachary whalen	1	0.0	634.0	0	24	632.0	
1620	zaheer khan	1	0.0	184.0	0	26	183.0	
1621	zhivago walker	1	0.0	126.0	0	30	125.0	
1622	zuberi floyd	1	0.0	757.0	0	22	756.0	

1623 rows × 9 columns

In [299…

```
plt.plot(times, ibs_by_time)
plt.axhline(y=mean_ibs, linestyle="--")
plt.title("IBS over Time")
plt.xlabel("Time")
plt.ylabel("Brier Score")
plt.show()
```



In [300…

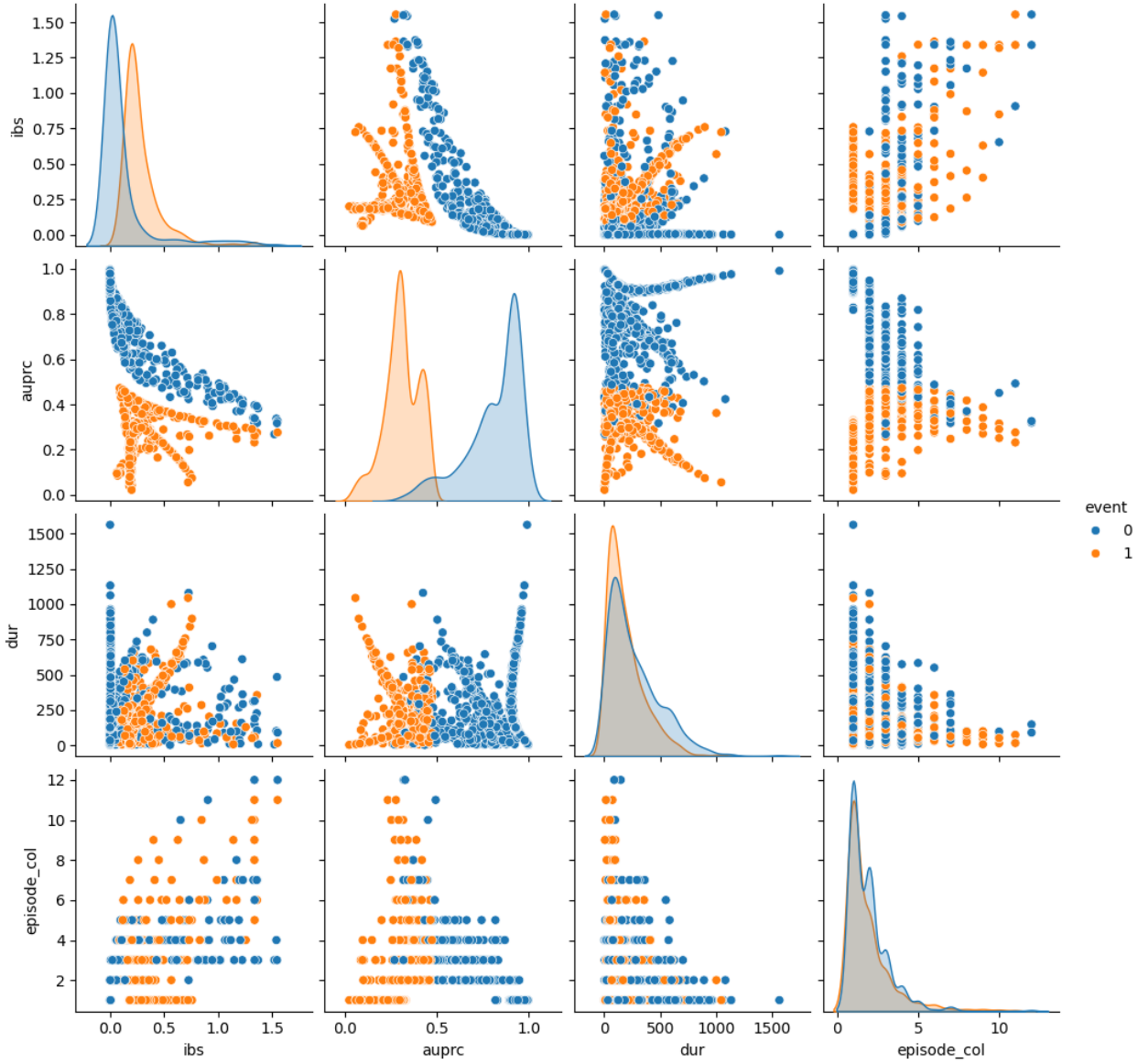
```
from survivors import metrics
test_df_results = test_df[["name", "episode_col", "dur", "event"]].copy()

test_df_results["dur"] = test_df["dur"] - test_df["entry"]

test_df_results["ibs"] = metrics.ibs(
            survival_train=survival_train,
            survival_test=survival_test,
            estimate=estimate,
            times=times,
            axis=0
        )
test_df_results["auprc"] = metrics.auprc(
            survival_train=survival_train_cens,
            survival_test=survival_test_cens,
            estimate=estimate,
            times=times,
            axis=0
        )
```

In [301…

```
# sns.scatterplot(x="episode_col", y="auprc", data=test_df_results)
sns.pairplot(test_df_results, vars=["ibs", "auprc", "dur", "episode_col"]
```

Out[301…

```
<seaborn.axisgrid.PairGrid at 0x2c4d48fa0>
```



In [302…

```
test_df_results[test_df_results["episode_col"] >= 8].round(3)
# test_df["dur"].describe()
```

Out[302…

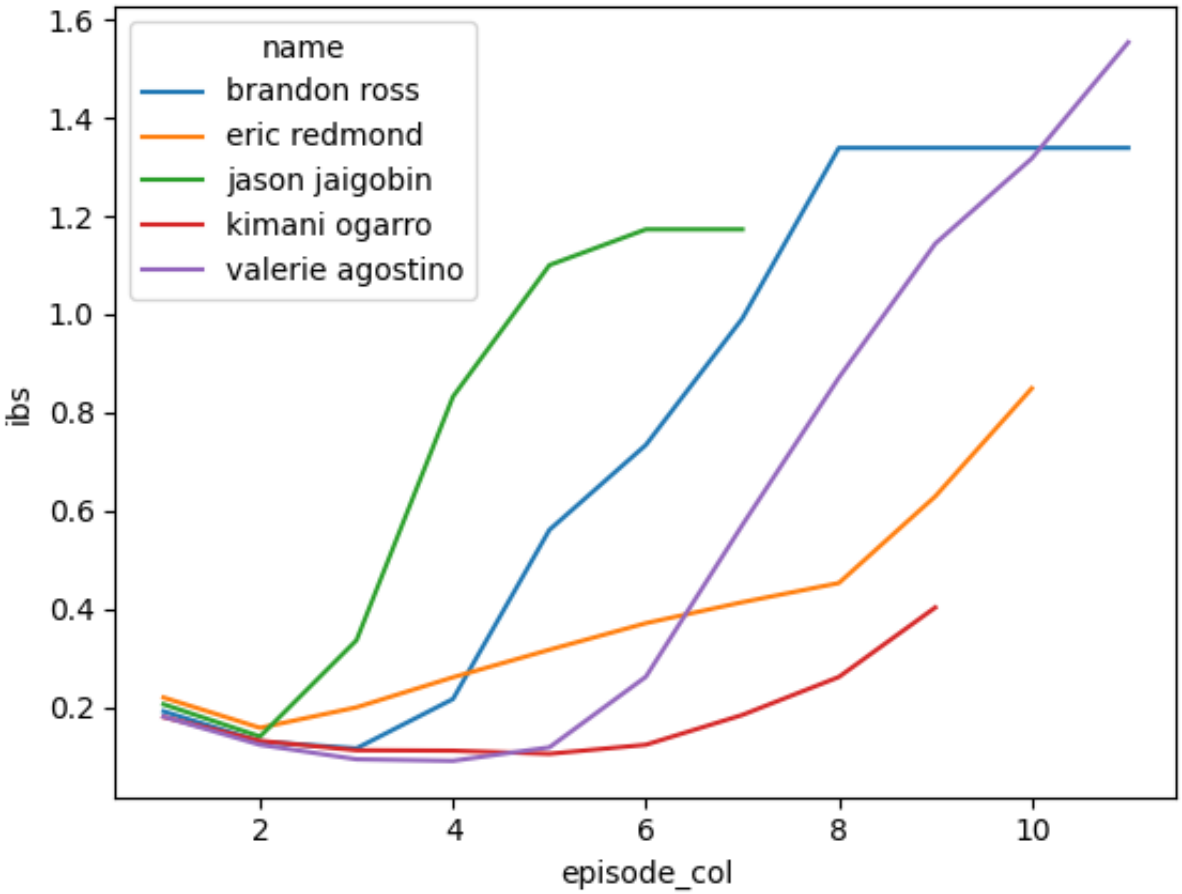
	name	episode_col	dur	event	ibs	auprc
216	brandon ross	8	47.0	1	1.339	0.288
217	brandon ross	9	47.0	1	1.339	0.270
218	brandon ross	10	67.0	1	1.339	0.251
219	brandon ross	11	74.0	1	1.339	0.231
220	brandon ross	12	148.0	0	1.339	0.319
592	eric redmond	8	23.0	1	0.453	0.364
593	eric redmond	9	100.0	1	0.629	0.340
594	eric redmond	10	14.0	1	0.849	0.315
595	eric redmond	11	18.0	0	0.907	0.492
770	jason jaigobin	8	99.0	0	1.173	0.370
1011	kimani ogarro	8	79.0	1	0.261	0.417
1012	kimani ogarro	9	74.0	1	0.403	0.387
1013	kimani ogarro	10	97.0	0	0.653	0.451
1554	valerie agostino	8	97.0	1	0.871	0.325
1555	valerie agostino	9	7.0	1	1.144	0.302
1556	valerie agostino	10	51.0	1	1.317	0.291
1557	valerie agostino	11	16.0	1	1.554	0.276
1558	valerie agostino	12	90.0	0	1.554	0.326

In [303…

```
freq_name = test_df_results.query("episode_col >= 8")["name"].unique()
sns.lineplot(x="episode_col", y="ibs", hue="name",
                data=test_df_results.query(f"name.isin({list(freq_name)})
```

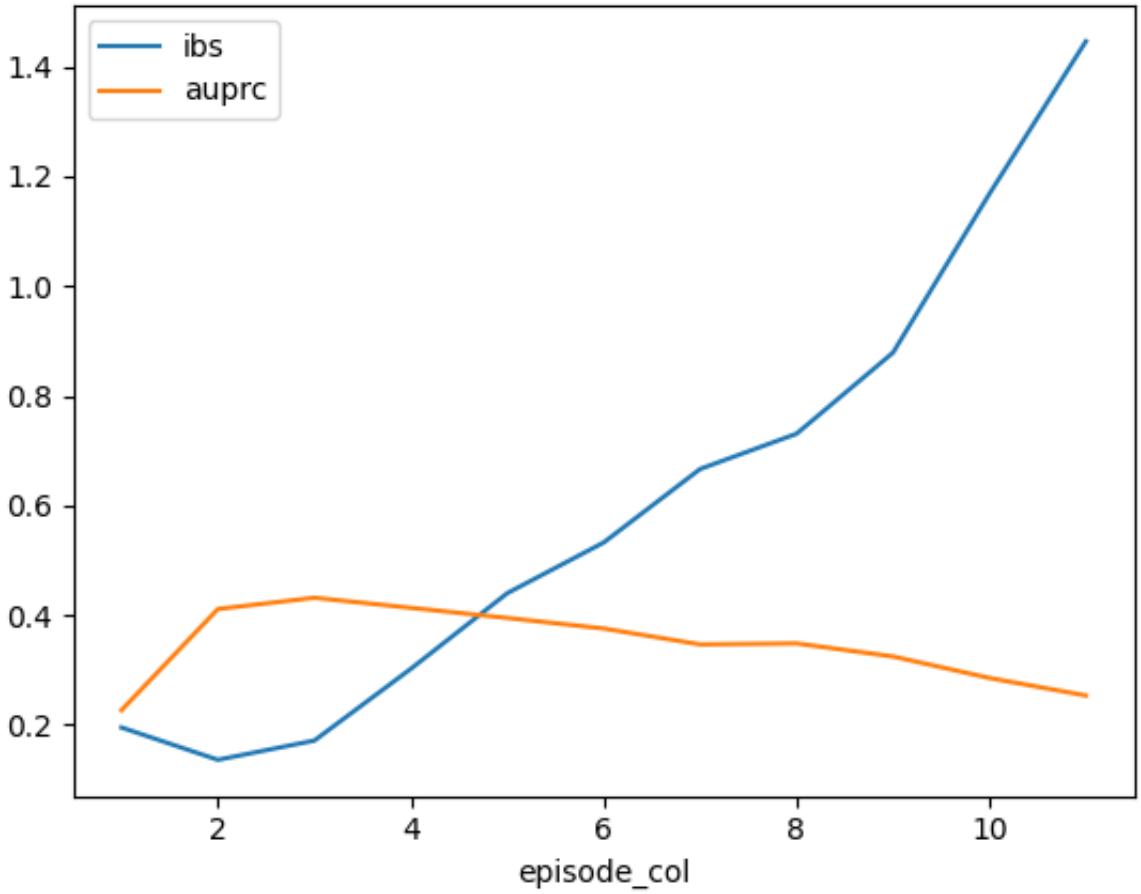
Out[303…

```
<Axes: xlabel='episode_col', ylabel='ibs'>
```



In [304…
```
test_df_results.query(f"name.isin({list(freq_name)}) and (event == 1)").g
```

Out[304…
```
<Axes: xlabel='episode_col'>
```



In [305…

```
# tr_pred = 1 - model.predict_survival(train_df, times).values.T
tr_pred = model.predict_cumulative_hazard(train_df, times)
tr_max = np.quantile(tr_pred.max(), 0.95)

# pred = 1 - model.predict_survival(test_df, times).values.T
# test_df.drop_duplicates(["name"], keep='first')
pred = model.predict_cumulative_hazard(test_df, times)
# pred[:] = train_df.groupby("name")["event"].sum().mean()  // baseline c
print("pred shape:", pred.shape)
print("len(test_df):", len(test_df))
metric = RecurrentCountError()

recurrent_error = metric.compute(
    survival_train=None,
    survival_test=test_df,
    estimate=pred / tr_max,
    times=times
)

mean_events = test_df.groupby("name")["event"].sum().mean()
print("Mean event =", mean_events)
print("Integrated cumulative event error:", recurrent_error)
```

```
pred shape: (200, 1623)
len(test_df): 1623
Mean event = 0.9207100591715977
Integrated cumulative event error: 0.2708145327042235
```

In [306…

```
pred
```

Out[306…

	0	1	2	3	4	5	
0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000
7.068342	0.000412	0.002918	0.003342	0.002031	0.000496	0.001035	0.00
14.136683	0.001425	0.010094	0.011557	0.007025	0.001717	0.003578	0.00
21.205025	0.002533	0.017942	0.020544	0.012487	0.003051	0.006361	0.00
28.273367	0.003430	0.024295	0.027818	0.016908	0.004132	0.008613	0.00
...	...	...	...	...	...	...	
1378.326633	0.269509	1.908951	2.185718	1.328514	0.324629	0.676736	0.67
1385.394975	0.269770	1.910795	2.187830	1.329798	0.324943	0.677389	0.67
1392.463317	0.271370	1.922128	2.200806	1.337685	0.326870	0.681407	0.68
1399.531658	0.272979	1.933529	2.213860	1.345619	0.328809	0.685449	0.685
1406.600000	0.272979	1.933529	2.213860	1.345619	0.328809	0.685449	0.685

200 rows × 1623 columns

In [307…

```
test_df.drop_duplicates(["name"], keep='first')
```

Out[307…

	name	episode_col	entry	dur	event	age	time_since_last_arrest	av
0	aaron hammond	1	0.0	699.0	0	25	688.0	
1	aaron villacampa	1	0.0	107.0	0	26	106.0	
2	abraham ballestas	1	0.0	35.0	0	52	34.0	
3	acie canteen	1	0.0	215.0	0	27	214.0	
4	adam gillstrap	1	0.0	630.0	0	35	628.0	
...	...	...	...	...	...	...	...	
1618	zachary payton	1	0.0	34.0	0	28	33.0	
1619	zachary whalen	1	0.0	634.0	0	24	632.0	
1620	zaheer khan	1	0.0	184.0	0	26	183.0	
1621	zhivago walker	1	0.0	126.0	0	30	125.0	
1622	zuberi floyd	1	0.0	757.0	0	22	756.0	

845 rows × 9 columns

Using the updated CoxModelUpd file (cox_model_upd.py). RECURRENT WITH EPISODE_COL case

In [308…

```
# RECURRENT WITH EPISODE_COL
features1 = [c for c in train_df.columns
            if c not in ["name", "entry", "time", "event", "dur"]]

cols_for_model1 = ["name", "episode_col", "entry", "time", "event"] + \
                  [c for c in features1 if c != "episode_col"]

train_model_df1 = train_df[cols_for_model1].copy()

model1 = CoxModelUpd(
    features=[c for c in features1 if c != "episode_col"],
    mode="recurrent",
    use_episode=True
)

model1.fit(train_model_df1)
```

model	lifelines.CoxPHFitter
duration col	'time'
event col	'event'
entry col	'entry'
cluster col	'name'
robust variance	True
strata	episode_col
baseline estimation	breslow
number of observations	6149
number of events observed	2767
partial log-likelihood	-16050.54
time fit was run	2026-03-04 12:44:31 UTC

	coef	exp(coef)	se(coef)	coef lower 95%	coef upper 95%	exp(coef) lower 95%	exp(coef uppe 95%
age	-0.01	0.99	0.00	-0.01	-0.00	0.99	1.0
time_since_last_arrest	-0.01	0.99	0.00	-0.02	-0.01	0.98	0.9
average_dur_custody	-0.01	0.99	0.00	-0.02	-0.01	0.98	0.9

Concordance	0.95
Partial AIC	32107.08
log-likelihood ratio test	4470.49 on 3 df
-log2(p) of ll-ratio test	inf

```
<class 'lifelines.fitters.coxph_fitter.CoxPHFitter'>
```

In [ ]:

```python
from lifelines.utils import concordance_index
#times = np.linspace(0, train_df["dur"].max(), 200)
train_dur1 = train_model_df1["time"]
horizon1 = np.quantile(train_dur1, 0.95)
times1 = np.linspace(0, horizon1, 200)

test_model_df1 = test_df[cols_for_model1].copy()

print("Train episodes:", sorted(train_model_df1["episode_col"].unique()))
print("Test episodes:", sorted(test_model_df1["episode_col"].unique()))

# had an issue with strata, so removing unseen strata
valid_episodes = train_model_df1["episode_col"].unique()
test_model_df1 = test_model_df1[
    test_model_df1["episode_col"].isin(valid_episodes)
```

```
]


predictions1 = model1.predict_survival(test_model_df1, times1)
#predictions = model.predict_survival(test_df, times)
estimate1 = predictions1.values.astype(np.float64)
times1 = np.asarray(times1, dtype=np.float64)

test_dur1 = test_model_df1["time"]

survival_test1 = np.array(
    list(zip(
        test_model_df1["event"].astype(bool),
        test_dur1.astype(float)
    )),
    dtype=[("event", "?"), ("time", "f8")]
)

survival_train1 = np.array(
    list(zip(
        train_model_df1["event"].astype(bool),
        train_dur1.astype(float)
    )),
    dtype=[("event", "?"), ("time", "f8")]
)
print(survival_test1.dtype.names)


ibs_metric1 = IBSMetric()
mean_ibs1, ibs_by_time1 = ibs_metric.compute(
    survival_train1,
    survival_test1,
    estimate1,
    times1
)

print(mean_ibs1)

survival_test_cens1 = np.array(
    list(zip(
        (~test_model_df1["event"].astype(bool)),
        test_dur1.astype(float)
    )),
    dtype=[("cens", "?"), ("time", "f8")]
)

survival_train_cens1 = np.array(
    list(zip(
        (~train_model_df1["event"].astype(bool)),
        train_dur1.astype(float)
    )),
    dtype=[("cens", "?"), ("time", "f8")]
)

auprc_metric1 = AUPRCMetric()
auprc1 = auprc_metric1.compute(
    survival_train_cens1,
```

```
    survival_test_cens1,
    estimate1,
    times1
)
print("AUPRC:", np.mean(auprc1))

ci_metric1 = concordance_index(
    test_dur1,
    np.trapz(predictions1.values, times1, axis=1),
    test_model_df1["event"]
)
print("Concordance Index:", ci_metric1)
```

```
Train episodes: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Test episodes: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
('event', 'time')
0.3763596913359819
AUPRC: 0.5562184289900506
Concordance Index: 0.5058826930915379
```

Using the updated CoxModelUpd file (cox_model_upd.py). RECURRENT WITHOUT EPISODE_COL case

In [310…

```
# RECURRENT WITHOUT EPISODE_COL
features2 = [c for c in train_df.columns
            if c not in ["name", "entry", "time", "event", "dur", "episod

cols_for_model2 = ["name", "entry", "time", "event"] + features
train_model_df2 = train_df[cols_for_model2].copy()

model2 = CoxModelUpd(
    features=features2,
    mode="recurrent",
    use_episode=False
)
model2.fit(train_model_df2)
```

model	lifelines.CoxPHFitter
duration col	'time'
event col	'event'
entry col	'entry'
cluster col	'name'
robust variance	True
baseline estimation	breslow
number of observations	6149
number of events observed	2767
partial log-likelihood	-20405.07
time fit was run	2026-03-04 12:44:32 UTC

	coef	exp(coef)	se(coef)	coef lower 95%	coef upper 95%	exp(coef) lower 95%	exp(coef upper 95%
age	-0.00	1.00	0.00	-0.01	-0.00	0.99	1.0
time_since_last_arrest	-0.00	1.00	0.00	-0.00	-0.00	1.00	1.0
average_dur_custody	-0.00	1.00	0.00	-0.00	-0.00	1.00	1.0

Concordance	0.74
Partial AIC	40816.15
log-likelihood ratio test	1166.16 on 3 df
-log2(p) of ll-ratio test	836.44

```
<class 'lifelines.fitters.coxph_fitter.CoxPHFitter'>
```

In [ ]:

```
from lifelines.utils import concordance_index

train_dur2 = train_model_df2["time"]
horizon2 = np.quantile(train_dur2, 0.95)
times2 = np.linspace(0, horizon2, 200)
times2 = np.asarray(times2, dtype=np.float64)

test_model_df2 = test_df[cols_for_model2].copy()

# reording to avoid issues
test_model_df2 = test_model_df2.reset_index(drop=True)

predictions2 = model2.predict_survival(test_model_df2, times2)
estimate2 = predictions2.values.astype(np.float64)

test_dur2 = test_model_df2["time"]
train_dur2 = train_model_df2["time"]
```

```python
survival_test2 = np.array(
    list(zip(
        test_model_df2["event"].astype(bool),
        test_dur2.astype(float)
    )),
    dtype=[("event", "?"), ("time", "f8")]
)

survival_train2 = np.array(
    list(zip(
        train_model_df2["event"].astype(bool),
        train_dur2.astype(float)
    )),
    dtype=[("event", "?"), ("time", "f8")]
)

ibs_metric2 = IBSMetric()
mean_ibs2, ibs_by_time2 = ibs_metric2.compute(
    survival_train2,
    survival_test2,
    estimate2,
    times2
)
print("IBS:", mean_ibs2)

survival_test_cens2 = np.array(
    list(zip(
        ~test_model_df2["event"].astype(bool),
        test_dur2.astype(float)
    )),
    dtype=[("cens", "?"), ("time", "f8")]
)

survival_train_cens2 = np.array(
    list(zip(
        ~train_model_df2["event"].astype(bool),
        train_dur2.astype(float)
    )),
    dtype=[("cens", "?"), ("time", "f8")]
)

auprc_metric2 = AUPRCMetric()
auprc2 = auprc_metric2.compute(
    survival_train_cens2,
    survival_test_cens2,
    estimate2,
    times2
)
print("AUPRC:", np.mean(auprc2))

ci_metric2 = concordance_index(
    test_dur2,
    np.trapz(estimate2, times2, axis=1),
    test_model_df2["event"]
)
print("Concordance Index:", ci_metric2)
```

```
IBS: 0.2054054924976589
AUPRC: 0.5484192062384199
Concordance Index: 0.7116620005947137
```

Using the updated CoxModelUpd file (cox_model_upd.py). INDEPENDENT case

In [312…

```python
# INDEPENDENT
train_ind = train_df.drop_duplicates("name").copy()
train_ind["duration"] = train_ind["time"] - train_ind["entry"]

features3 = [c for c in train_ind.columns
            if c not in ["name", "entry", "time", "duration", "event", "d

cols_for_model3 = ["duration", "event"] + features3
train_model_df3 = train_ind[cols_for_model3].copy()

model3 = CoxModelUpd(
    features=features3,
    mode="independent"
)

model3.fit(train_model_df3)
```

```
/Library/Frameworks/Python.framework/Versions/3.10/lib/python3.10/site-pac
kages/lifelines/fitters/coxph_fitter.py:1614: ConvergenceWarning: Newton-R
aphson failed to converge sufficiently. Please see the following tips in t
he lifelines documentation: https://lifelines.readthedocs.io/en/latest/Exa
mples.html#problems-with-convergence-in-the-cox-proportional-hazard-model
  warnings.warn(
```

model	lifelines.CoxPHFitter
duration col	'duration'
event col	'event'
baseline estimation	breslow
number of observations	3382
number of events observed	1503
partial log-likelihood	-7574.77
time fit was run	2026-03-04 12:44:33 UTC

	coef	exp(coef)	se(coef)	coef lower 95%	coef upper 95%	exp(coef) lower 95%	exp(coef upper 95%
age	-0.01	0.99	0.00	-0.01	-0.00	0.99	1.0
time_since_last_arrest	-0.03	0.97	0.00	-0.04	-0.03	0.96	0.9
average_dur_custody	-0.03	0.97	0.00	-0.04	-0.03	0.97	0.9

Concordance	1.00
Partial AIC	15155.53
log-likelihood ratio test	7152.59 on 3 df
-log2(p) of ll-ratio test	inf

```
<class 'lifelines.fitters.coxph_fitter.CoxPHFitter'>
```

In [ ]:

```python
from lifelines.utils import concordance_index

# doing one row per individual
test_ind = test_df.drop_duplicates("name").copy()

# duration = time - entry
test_ind["duration"] = test_ind["time"] - test_ind["entry"]

test_model_df3 = test_ind[cols_for_model3].copy()
test_model_df3 = test_model_df3.reset_index(drop=True)

train_dur3 = train_model_df3["duration"]
horizon3 = np.quantile(train_dur3, 0.95)

times3 = np.linspace(0, horizon3, 200)
times3 = np.asarray(times3, dtype=np.float64)

predictions3 = model3.predict_survival(test_model_df3, times3)
estimate3 = predictions3.values.astype(np.float64)

test_dur3 = test_model_df3["duration"]
train_dur3 = train_model_df3["duration"]
```

```python
survival_test3 = np.array(
    list(zip(
        test_model_df3["event"].astype(bool),
        test_dur3.astype(float)
    )),
    dtype=[("event", "?"), ("time", "f8")]
)

survival_train3 = np.array(
    list(zip(
        train_model_df3["event"].astype(bool),
        train_dur3.astype(float)
    )),
    dtype=[("event", "?"), ("time", "f8")]
)

ibs_metric3 = IBSMetric()
mean_ibs3, ibs_by_time3 = ibs_metric3.compute(
    survival_train3,
    survival_test3,
    estimate3,
    times3
)
print("IBS:", mean_ibs3)

survival_test_cens3 = np.array(
    list(zip(
        ~test_model_df3["event"].astype(bool),
        test_dur3.astype(float)
    )),
    dtype=[("cens", "?"), ("time", "f8")]
)

survival_train_cens3 = np.array(
    list(zip(
        ~train_model_df3["event"].astype(bool),
        train_dur3.astype(float)
    )),
    dtype=[("cens", "?"), ("time", "f8")]
)

auprc_metric3 = AUPRCMetric()
auprc3 = auprc_metric3.compute(
    survival_train_cens3,
    survival_test_cens3,
    estimate3,
    times3
)
print("AUPRC:", np.mean(auprc3))

ci_metric3 = concordance_index(
    test_dur3,
    np.trapz(estimate3, times3, axis=1),
    test_model_df3["event"]
)
print("Concordance Index:", ci_metric3)
```

```
IBS: 0.012534467749523482
AUPRC: 0.849836063390143
Concordance Index: 0.9954563281391989
```